

GraphQL সম্পূর্ণ গাইড — Basic থেকে Advanced (MERN Developer-দের জন্য)

তুমি যেহেতু MERN স্ট্যাক জানো (MongoDB, Express, React, Node), তাই প্রতিটা কনসেপ্ট REST API আর Express-এর সাথে তুলনা করে বোঝানো হয়েছে। শেষের দিকে Prisma ORM আর relational database (PostgreSQL/MySQL) দিয়ে কীভাবে GraphQL ব্যবহার করবে, সেটাও আলাদা সেকশনে আছে।

সূচিপত্র

1. GraphQL কী এবং কেন দরকার
 2. REST vs GraphQL — পার্থক্য
 3. মূল বিল্ডিং ব্লক: Schema, Types, Query, Mutation
 4. GraphQL Server সেটআপ (Apollo Server + Express)
 5. Resolver — গভীরভাবে বোঝা
 6. MongoDB/Mongoose-এর সাথে কানেক্ট করা
 7. React-এ Apollo Client ব্যবহার
 8. Advanced: N+1 Problem এবং DataLoader
 9. Advanced: Pagination (Offset vs Cursor-based)
 10. Advanced: Authentication ও Authorization
 11. Advanced: Error Handling
 12. Advanced: Input Types, Validation, File Upload
 13. Advanced: Subscriptions (Real-time)
 14. Advanced: Schema Design Best Practices, Fragments
 15. Advanced: Security (Query Depth, Complexity, Rate Limit)
 16. Advanced: Apollo Federation (Microservices)
 17. Testing GraphQL API
 18. Prisma ORM + Relational Database দিয়ে GraphQL (ভবিষ্যতের জন্য প্রস্তুতি)
 19. Full Mini Project: Blog API
 20. শেষার Roadmap এবং Resources
-

১. GraphQL কী এবং কেন দরকার

GraphQL হলো একটা **query language for APIs**, যেটা Facebook ২০১২ সালে তৈরি করেছিল নিজেদের mobile app-এর performance সমস্যা সমাধানের জন্য। এটা কোনো ডেটাবেজ না, কোনো ফ্রেমওয়ার্কও না — এটা তোমার API-কে ডেটা রিকোয়েস্ট করার একটা নিয়ম (specification)।

REST-এ তুমি যেটা অভ্যস্ত: সার্ভার ঠিক করে দেয় প্রতিটা endpoint কী ডেটা রিটার্ন করবে। ক্লায়েন্ট চাইলেও কম বা বেশি ডেটা নিতে পারে না।

GraphQL-এ উল্টোটা: ক্লায়েন্ট বলে দেয় ঠিক কোন কোন field তার দরকার, সার্ভার ঠিক সেই shape-এর ডেটাই রিটার্ন করে — না কম, না বেশি।

আসল সমস্যা যেটা GraphQL সমাধান করে

ধরো তোমার একটা Express REST API আছে:

```
GET /api/users/:id          -> user-এর basic info
GET /api/users/:id/posts    -> user-এর posts
GET /api/posts/:id/comments -> post-এর comments
```

একটা ইউজার প্রোফাইল পেজে যদি user info + তার শেষ ৩টা পোস্ট + প্রতিটা পোস্টের কमेंট কাউন্ট দেখাতে হয়, তাহলে তোমাকে **৩-৪টা আলাদা API call** করতে হবে (যাকে বলে **under-fetching**), অথবা তোমাকে একটা বিশাল `/api/users/:id/full-profile` endpoint বানাতে হবে যেটা সব ডেটা একসাথে পাঠায়, এমনকি যেসব ফিল্ড দরকার নেই সেগুলোও (যাকে বলে **over-fetching**)।

GraphQL-এ এই পুরো জিনিসটা **একটা মাত্র request**-এ হয়ে যায়, আর ক্লায়েন্ট ঠিক করে দেয় কী কী চাই:

```
graphql

query {
  user(id: "123") {
    name
    avatar
    posts(limit: 3) {
      title
      commentCount
    }
  }
}
```

২. REST vs GraphQL — পার্থক্য

বিষয়	REST	GraphQL
Endpoint	একাধিক endpoint (<code>/users</code> , <code>/posts</code>)	একটাই endpoint (<code>/graphql</code>)
ডেটার পরিমাণ	সার্ভার ঠিক করে	ক্লায়েন্ট ঠিক করে
HTTP Method	GET, POST, PUT, DELETE	সবসময় POST (সাধারণত)
HTTP Status	200, 404, 500 ইত্যাদি অর্থবহ	প্রায় সবসময় 200, error আলাদা ফিল্ডে আসে

বিষয়	REST	GraphQL
Versioning	<code>/api/v1</code> , <code>/api/v2</code>	দরকার পড়ে না, schema evolve করে
Caching	HTTP caching সহজ (URL-based)	জটিল, client-side cache লাগে (Apollo cache)
Over/Under-fetching	সমস্যা থাকে	থাকে না

গুরুত্বপূর্ণ কথা: GraphQL সবসময় REST-এর চেয়ে ভালো না। ছোট প্রজেক্টে, সিম্পল CRUD অ্যাপে REST এখনো বেশি সহজ এবং দ্রুত implement করা যায়। GraphQL মূলত কাজে লাগে যখন: একাধিক ক্লায়েন্ট (mobile + web) ভিন্ন ভিন্ন ডেটা চায়, ডেটা সম্পর্ক জটিল (nested relationships), অথবা over/under-fetching বাস্তব সমস্যা তৈরি করছে।

৩. মূল বিল্ডিং ব্লক: Schema, Types, Query, Mutation

Schema — তোমার API-এর contract

Schema হলো GraphQL API-এর হার্ট। এটা **SDL (Schema Definition Language)** দিয়ে লেখা হয়, আর বলে দেয় কী কী ডেটা পাওয়া যাবে এবং কী shape-এ।

```
graphql

type User {
  id: ID!
  name: String!
  email: String!
  age: Int
  isActive: Boolean!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  content: String!
  author: User!
  createdAt: String!
}
```

Scalar Types (বেসিক ডেটা টাইপ)

GraphQL-এ ৫টা built-in scalar type আছে:

- `Int` — পূর্ণসংখ্যা (32-bit)

- `Float` — দশমিক সংখ্যা
- `String` — টেক্সট
- `Boolean` — true/false
- `ID` — ইউনিক আইডেন্টিফায়ার (`String`-এর মতোই কাজ করে, কিন্তু সিমেন্টিক্যালি আলাদা)

❗ চিহ্নের মানে হলো **non-nullable** — মানে এই ফিল্ড কখনো `null` হতে পারবে না (Mongoose-এর `required: true`-এর মতো)। `[Post!]!` মানে — একটা array, যেটা নিজে null হবে না, আর array-এর ভেতরের কোনো element-ও null হবে না।

Query Type — ডেটা পড়া (READ)

`Query` টাইপ হলো তোমার সব GET request-এর জায়গা:

```
graphql

type Query {
  users: [User!]!
  user(id: ID!): User
  posts(limit: Int, search: String): [Post!]!
}
```

ক্লায়েন্ট থেকে এভাবে কল করা হয়:

```
graphql

query {
  user(id: "1") {
    name
    email
    posts {
      title
    }
  }
}
```

Mutation Type — ডেটা পরিবর্তন (CREATE, UPDATE, DELETE)

REST-এ POST/PUT/PATCH/DELETE যা করত, GraphQL-এ সব `Mutation`-এর ভেতরে যায়:

```
graphql

type Mutation {
  createUser(name: String!, email: String!): User!
  updateUser(id: ID!, name: String): User!
  deleteUser(id: ID!): Boolean!
}
```

```
graphql

mutation {
  createUser(name: "Rafi", email: "rafi@example.com") {
    id
    name
  }
}
```

লক্ষ্য করো — mutation-এও তুমি বলে দিতে পারছো রেসপন্সে কী কী field ফেরত চাও (এখানে `id` আর `name`)। এটাই GraphQL-এর সুপারপাওয়ার — Read আর Write দুটোতেই same flexibility।

Input Types

যখন একটা mutation-এ অনেক argument পাঠাতে হয়, তখন সেগুলোকে একটা `input` টাইপে গুছিয়ে নেওয়া হয় (অনেকটা Mongoose schema বানানোর মতো):

```
graphql

input CreateUserInput {
  name: String!
  email: String!
  age: Int
}

type Mutation {
  createUser(input: CreateUserInput!): User!
}
```

8. GraphQL Server সেটআপ (Apollo Server + Express)

তোমার এক্সিস্টিং Express অ্যাপের পাশে এভাবে GraphQL বসানো যায়:

```
bash

npm install @apollo/server graphql express mongoose
```

```
js
```

```
// server.js
import express from 'express';
import { ApolloServer } from '@apollo/server';
import { expressMiddleware } from '@apollo/server/express4';
import mongoose from 'mongoose';
import { typeDefs } from './schema.js';
import { resolvers } from './resolvers.js';

await mongoose.connect(process.env.MONGO_URI);

const app = express();

const apolloServer = new ApolloServer({ typeDefs, resolvers });
await apolloServer.start();

app.use(
  '/graphql',
  express.json(),
  expressMiddleware(apolloServer, {
    context: async ({ req }) => {
      // এখানে auth token verify করে user বের করতে পারো,
      // ঠিক যেভাবে তুমি Express middleware-এ req.user সেট করো
      return { user: req.user };
    },
  })
);

// তোমার পুরোনো REST routes এখানেও থাকতে পারে — দুটো একসাথে চলতে পারে
app.use('/api', restRoutes);

app.listen(4000, () => console.log('Server ready at http://localhost:4000/graphql'))
```

`/graphql` এ যাওয়ার পর Apollo নিজে থেকেই **Apollo Sandbox** নামে একটা GUI দেয় (Postman-এর মতোই, কিন্তু GraphQL-স্পেসিফিক), যেখানে তুমি query/mutation টেস্ট করতে পারবে আর schema explore করতে পারবে।

৫. Resolver — গভীরভাবে বোঝা

Resolver হলো একটা ফাংশন যেটা বলে দেয় schema-র একটা নির্দিষ্ট field-এর ডেটা কোথা থেকে এবং কীভাবে আসবে। এটা অনেকটা তোমার Express route handler-এর মতোই, পার্থক্য হলো — এটা পুরো route-এর বদলে একটা single field-এর জন্য কাজ করে।

Resolver-এর ৪টা argument

প্রতিটা resolver ফাংশন ৪টা argument পায়:

```
js  
  
fieldName: (parent, args, context, info) => { ... }
```

- **parent** — উপরের লেভেলের resolver যা রিটার্ন করেছে (nested resolver-এ এটা খুব গুরুত্বপূর্ণ)
- **args** — query/mutation-এ পাঠানো argument গুলো
- **context** — প্রতিটা request-এ shared একটা object (auth user, db connection ইত্যাদি রাখার জায়গা)
- **info** — query-র internal structure সম্পর্কে তথ্য (কম ব্যবহৃত হয়)

```
js  
  
export const resolvers = {  
  Query: {  
    users: async () => await User.find(),  
    user: async (_, { id }) => await User.findById(id),  
  },  
  
  Mutation: {  
    createUser: async (_, { input }) => {  
      const user = new User(input);  
      return await user.save();  
    },  
  },  
  
  // এটাই সবচেয়ে গুরুত্বপূর্ণ অংশ - relationship resolver  
  User: {  
    posts: async (parent) => {  
      // parent এখানে সেই User object, যেটা উপরের Query.user resolver রিটার্ন করেছিল  
      return await Post.find({ author: parent.id });  
    },  
  },  
};
```

কেন `User.posts` resolver-টা গুরুত্বপূর্ণ

REST-এ "ইউজার আর তার পোস্ট" একসাথে দেখাতে হলে তোমাকে হয় সবসময় posts embed করতে হতো, নাহলে আলাদা API call করতে হতো। GraphQL-এ এই সম্পর্কটা শুধু আরেকটা resolver — আর এটা শুধু তখনই রান হবে যখন ক্লায়েন্ট আসলেই `posts` field চেয়েছে। কেউ যদি শুধু `name` চায়, এই resolver একদম রান-ই হবে না। এটাই GraphQL-এর lazy resolution — পারফরম্যান্সের জন্য খুব গুরুত্বপূর্ণ।

৬. MongoDB/Mongoose-এর সাথে কানেক্ট করা

তোমার এক্সিস্টিং Mongoose মডেল প্রায় হুবহু একই থাকে, শুধু resolver-এর ভেতর থেকে কল হয়:

js

```
// models/User.js – এটা তোমার আগের মতোই
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
});

export const User = mongoose.model('User', userSchema);
```

graphql

```
# schema.graphql
type User {
  id: ID!
  name: String!
  email: String!
}

type Query {
  users: [User!]!
}
```

একটা সুস্থ জিনিস খেয়াল রাখবে — MongoDB-তে আইডি ফিল্ডের নাম (`_id`), কিন্তু GraphQL convention অনুযায়ী আমরা (`id`) ব্যবহার করি। Apollo Server এটা automatic handle করে না, তাই হয় resolver-এ map করতে হবে:

js

```
User: {
  id: (parent) => parent._id.toString(),
},
```

অথবা Mongoose schema-তে (`toJSON`) virtual সেট করে দিতে পারো যাতে (`_id`) সবসময় (`id`)-তে কনভার্ট হয়।

৭. React-এ Apollo Client ব্যবহার

bash


```
npm install @apollo/client graphql
```

```
jsx

// main.jsx
import { ApolloClient, InMemoryCache, ApolloProvider } from '@apollo/client';

const client = new ApolloClient({
  uri: 'http://localhost:4000/graphql',
  cache: new InMemoryCache(),
});

ReactDOM.createRoot(document.getElementById('root')).render(
  <ApolloProvider client={client}>
    <App />
  </ApolloProvider>
);
```

useQuery — ডেটা পড়া

```
jsx
```

```

import { useQuery, gql } from '@apollo/client';

const GET_USERS = gql`
  query GetUsers {
    users {
      id
      name
      email
    }
  }
`;

function UserList() {
  const { loading, error, data } = useQuery(GET_USERS);

  if (loading) return <p>লোড হচ্ছে...</p>;
  if (error) return <p>সমস্যা হয়েছে: {error.message}</p>;

  return (
    <ul>
      {data.users.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}

```

লক্ষ্য করো — `loading`, `error`, `data` — এই তিনটা state তুমি আগে `useEffect` + `useState` দিয়ে নিজে ম্যানেজ করতে। Apollo এটা ফ্রিতে দিয়ে দেয়।

useMutation — ডেটা লেখা

jsx

```
import { useMutation, gql } from '@apollo/client';

const CREATE_USER = gql`
  mutation CreateUser($input: CreateUserInput!) {
    createUser(input: $input) {
      id
      name
    }
  }
`;

function CreateUserForm() {
  const [createUser, { loading }] = useMutation(CREATE_USER, {
    refetchQueries: [GET_USERS], // create হওয়ার পর list অটো রিফ্রেশ
  });

  const handleSubmit = (e) => {
    e.preventDefault();
    createUser({ variables: { input: { name: 'Rafi', email: 'rafi@x.com' } } });
  };

  return <button onClick={handleSubmit} disabled={loading}>তৈরি করো</button>;
}
```

Apollo Cache — Redux-এর বিকল্প (অনেকটা)

Apollo Client একটা **normalized cache** রাখে। মানে দুইটা আলাদা কম্পোনেন্ট যদি একই `User`-এর ডেটা চায়, তারা cache থেকে শেয়ার করে — আর একটা জায়গায় update হলে সব জায়গায় reflect হয়। অনেক ক্ষেত্রে এটা Redux দিয়ে server-state ম্যানেজ করার দরকার কমিয়ে দেয়।

৮. Advanced: N+1 Problem এবং DataLoader

এটা GraphQL-এর সবচেয়ে কুখ্যাত পারফরম্যান্স সমস্যা, প্রত্যেক GraphQL ডেভেলপারকেই একদিন এর মুখোমুখি হতে হয়।

সমস্যাটা কী

graphql

```
query {  
  users {  
    name  
    posts { title }  
  }  
}
```

যদি ১০০ জন user থাকে, আর `User.posts` resolver প্রতিটা user-এর জন্য আলাদা `Post.find({ author: id })` কল করে, তাহলে মোট হবে:

- ১টা query (সব users আনার জন্য)
- • ১০০টা query (প্রতিটা user-এর posts আনার জন্য)

= ১০১টা ডেটাবেজ কল, যেখানে আদর্শভাবে ২টাতেই হওয়া উচিত ছিল। এটাকেই বলে **N+1 problem**।

সমাধান: DataLoader

DataLoader একটা লাইব্রেরি যেটা একই "tick"-এ আসা সব request **batch** করে নেয়, আর একটাই কুয়েরিতে সব ডেটা নিয়ে আসে।

```
bash
```

```
npm install dataloader
```

```
js
```

```
import DataLoader from 'dataloader';
```

```
const postsByUserLoader = new DataLoader(async (userIds) => {  
  // ১০০টা আলাদা কলের বদলে, একটাই কুয়েরি - $in দিয়ে  
  const posts = await Post.find({ author: { $in: userIds } });  
  
  // প্রতিটা userId-এর জন্য তার posts গ্রুপ করে রিটার্ন করা লাগবে,  
  // ঠিক সেই order-এই যে order-এ userIds এসেছিল  
  return userIds.map((id) =>  
    posts.filter((p) => p.author.toString() === id.toString())  
  );  
});
```

```
// resolver-এ ব্যবহার
```

```
User: {  
  posts: (parent, _, context) => context.loaders.postsByUserLoader.load(parent.  
},
```

Loader প্রতি request-এ নতুন করে বানাতে হয় (context-এ), যাতে এক ইউজারের ডেটা আরেক request-এ লিক না করে:

```
js

context: async ({ req }) => ({
  user: req.user,
  loaders: { postsByUserLoader: createPostsByUserLoader() },
}),
```

৯. Advanced: Pagination (Offset vs Cursor-based)

`users: [User!]!` সব ইউজার একসাথে রিটার্ন করা — ছোট ডেটাসেটে ঠিক আছে, কিন্তু production-এ এটা বিপজ্জনক (লক্ষ লক্ষ row একসাথে আনতে চাইলে সার্ভার ক্র্যাশ করবে)।

Offset-based (REST-এর মতোই পরিচিত)

```
graphql

type Query {
  users(offset: Int, limit: Int): [User!]!
}
```

সহজ, কিন্তু সমস্যা আছে — মাঝখানে যদি নতুন ডেটা insert/delete হয়, তাহলে পেজ skip বা duplicate হতে পারে।

Cursor-based (GraphQL-এর idiomatic উপায় — Relay Connection Spec)

```
graphql
```

```

type UserConnection {
  edges: [UserEdge!]!
  pageInfo: PageInfo!
}

type UserEdge {
  node: User!
  cursor: String!
}

type PageInfo {
  hasNextPage: Boolean!
  endCursor: String
}

type Query {
  users(first: Int!, after: String): UserConnection!
}

```

```

js

users: async (_, { first, after }) => {
  const query = after ? { _id: { $gt: after } } : {};
  const users = await User.find(query).limit(first + 1).sort({ _id: 1 });

  const hasNextPage = users.length > first;
  const result = hasNextPage ? users.slice(0, -1) : users;

  return {
    edges: result.map((u) => ({ node: u, cursor: u._id.toString() })),
    pageInfo: {
      hasNextPage,
      endCursor: result.length ? result[result.length - 1]._id.toString() : null
    },
  };
},

```

এটা প্রথমে একটু জটিল লাগবে, কিন্তু এটাই বড় অ্যাপগুলো (GitHub API, Facebook) ব্যবহার করে কারণ এটা scroll/infinite-load-এর জন্য বেশি স্থিতিশীল (stable)।

১০. Advanced: Authentication ও Authorization

Express-এ তুমি route-এর আগে middleware বসাতে — GraphQL-এ একটাই endpoint, তাই middleware-চেইন কাজ করে না একইভাবে। দুটো জায়গায় auth হ্যান্ডল হয়:

১. Context-এ user বসানো (একবারই, প্রতি request-এ)

```
js

context: async ({ req }) => {
  const token = req.headers.authorization?.replace('Bearer ', '');
  const user = token ? await verifyToken(token) : null;
  return { user };
},
```

২. প্রতিটা resolver-এ চেক করা

```
js

Mutation: {
  deletePost: async (_, { id }, context) => {
    if (!context.user) {
      throw new GraphQLError('আপনি লগইন করেননি', {
        extensions: { code: 'UNAUTHENTICATED' },
      });
    }

    const post = await Post.findById(id);
    if (post.author.toString() !== context.user.id) {
      throw new GraphQLError('এই পোস্ট ডিলিট করার অনুমতি নেই', {
        extensions: { code: 'FORBIDDEN' },
      });
    }

    await post.deleteOne();
    return true;
  },
},
```

বড় প্রজেক্টে প্রতিটা resolver-এ এই কোড বারবার লেখার বদলে **schema directive** দিয়ে কাজটা ক্লিন করা যায় (যেমন `@auth` ডিরেক্টিভ), কিন্তু শুরুতে resolver-এ ম্যানুয়াল চেক দিয়েই শুরু করো — এটা সহজে বোঝা যায়।

১১. Advanced: Error Handling

এখানে REST থেকে সবচেয়ে বড় পার্থক্যটা আসে: **GraphQL সবসময় HTTP 200 রিটার্ন করে**, এমনকি যখন কিছু ভুল হয়! এরর গুলো response-এর `errors` array-তে আসে, `data`-এর পাশাপাশি:

```
json
```

```
{
  "data": { "user": null },
  "errors": [
    {
      "message": "User not found",
      "extensions": { "code": "NOT_FOUND" }
    }
  ]
}
```

মানে তুমি আর `response.status === 404` চেক করে এরর ধরতে পারবে না — Apollo Client-এ এটা `error` object-এর ভেতর আসে (আগের `useQuery` উদাহরণে দেখেছ)।

কাস্টম এরর থ্রো করতে:

```
js
import { GraphQLError } from 'graphql';

throw new GraphQLError('ইউজার পাওয়া যায়নি', {
  extensions: {
    code: 'NOT_FOUND',
    http: { status: 404 }, // চাইলে actual HTTP status-ও সেট করা যায়
  },
});
```

১২. Advanced: Input Types, Validation, File Upload

Validation কোথায় হবে

GraphQL schema টাইপ-চেকিং করে (যেমন `String!` মানে এটা string হতেই হবে, missing হলে চলবে না), কিন্তু বিজনেস validation (যেমন email format, password length) এখনও resolver বা একটা আলাদা validation library (zod, yup) দিয়েই করতে হয় — ঠিক যেভাবে তুমি Express-এ করতে।

```
js
```



```
import { z } from 'zod';

const createUserSchema = z.object({
  email: z.string().email(),
  name: z.string().min(2),
});

createUser: async (_, { input }) => {
  createUserSchema.parse(input); // ভুল হলে throw করবে
  const user = new User(input);
  return await user.save();
},
```

File Upload

GraphQL স্পেসিফিকেশনে file upload বিল্ট-ইন নেই (যেহেতু GraphQL মূলত JSON-ভিত্তিক)। সবচেয়ে সাধারণ প্যাটার্ন: একটা আলাদা REST endpoint দিয়ে ফাইল আপলোড করে URL পাওয়া, তারপর সেই URL-টা GraphQL mutation-এ পাঠানো (যেমন S3/Cloudinary URL)। `graphql-upload` লাইব্রেরি দিয়ে সরাসরি multipart upload-ও করা যায়, কিন্তু বেশিরভাগ প্রোডাকশন অ্যাপ আলাদা আপলোড endpoint-ই বেছে নেয় — সহজ এবং কম জটিল।

১৩. Advanced: Subscriptions (Real-time)

Socket.io দিয়ে তুমি যা করতে, GraphQL-এ সেটার সমান্তরাল হলো **Subscription** — WebSocket-এর উপর চলে।

```
graphql

type Subscription {
  postCreated: Post!
}
```

```
js
```

```
import { PubSub } from 'graphql-subscriptions';
const pubsub = new PubSub();

const resolvers = {
  Mutation: {
    createPost: async (_, { input }) => {
      const post = await Post.create(input);
      pubsub.publish('POST_CREATED', { postCreated: post });
      return post;
    },
  },
  Subscription: {
    postCreated: {
      subscribe: () => pubsub.asyncIterator(['POST_CREATED']),
    },
  },
};
```

ক্লায়েন্টে:

```
jsx

const POST_CREATED = gql`
  subscription {
    postCreated { id title }
  }
`;

const { data } = useSubscription(POST_CREATED);
```

production-এ `graphql-ws` লাইব্রেরি দিয়ে WebSocket সার্ভার আলাদাভাবে সেটআপ করতে হয়, আর একাধিক সার্ভার ইনস্ট্যান্স থাকলে `PubSub`-এর জায়গায় Redis-backed pubsub (যেমন `graphql-redis-subscriptions`) লাগবে — in-memory `PubSub` শুধু single-instance dev/small app-এর জন্য ঠিক আছে।

১৪. Advanced: Schema Design Best Practices, Fragments

কিছু জিনিস যেগুলো বড় GraphQL প্রজেক্টে ফারাক গড়ে দেয়:

Database schema আর GraphQL schema এক না। Mongoose মডেল হুবহু কপি করে GraphQL টাইপ বানিও না — GraphQL schema হওয়া উচিত ক্লায়েন্টের প্রয়োজন অনুযায়ী ডিজাইন করা (client-driven), ডেটাবেজের গঠন অনুযায়ী না (database-driven)।

Mutation-এর নাম action-based রাখো: `createUser`, `updateUserEmail`, `deactivateUser` — শুধু একটা generic `updateUser(everything)` বানানোর বদলে নির্দিষ্ট mutation বানানো ভালো

অভ্যাস, কারণ এতে কে কী পরিবর্তন করতে চাইছে সেটা স্পষ্ট হয় এবং authorization সহজ হয়।

Fragments দিয়ে repeated field সেট পুনর্ব্যবহার করো:

```
graphql

fragment UserBasicInfo on User {
  id
  name
  avatar
}

query {
  users { ...UserBasicInfo }
  currentUser { ...UserBasicInfo email }
}
```

১৫. Advanced: Security (Query Depth, Complexity, Rate Limit)

REST-এ এই সমস্যাগুলো ছিলই না, কারণ প্রতিটা endpoint fixed কাজ করত। GraphQL-এ ক্লায়েন্ট নিজেই query গঠন করতে পারে বলে নতুন একটা ঝুঁকি তৈরি হয়:

```
graphql

query EvilQuery {
  users {
    posts {
      author {
        posts {
          author {
            posts { ... } # এভাবে অনেক গভীর পর্যন্ত যেতে পারে
          }
        }
      }
    }
  }
}
```

এই ধরনের deeply-nested query সার্ভারকে ওভারলোড করে ফেলতে পারে। সমাধান:

- **Query depth limiting** (`graphql-depth-limit`) — কতটুকু nested হতে পারবে তার সীমা
- **Query complexity analysis** (`graphql-query-complexity`) — প্রতিটা field-এর একটা "কস্ট" থাকে, মোট কস্ট একটা লিমিটের বেশি হলে রিজেক্ট
- **Rate limiting** — IP বা user অনুযায়ী, ঠিক REST-এর মতোই (`express-rate-limit`) GraphQL endpoint-এও কাজ করে)

- **Production-এ introspection বন্ধ রাখা** — যাতে বাইরের কেউ পুরো schema ঘেঁটে দেখতে না পারে

১৬. Advanced: Apollo Federation (Microservices)

যখন একটা টিম/প্রজেক্ট বড় হয়ে যায়, একটাই বিশাল schema ম্যানেজ করা কঠিন হয়ে পড়ে। Apollo Federation দিয়ে schema-কে একাধিক ছোট সার্ভিসে ভাগ করা যায় (Users service, Posts service, Orders service — প্রতিটা আলাদা GraphQL server), আর একটা **Gateway** এগুলোকে একটা unified schema হিসেবে দেখায় ক্লায়েন্টের কাছে। এটা মূলত microservices আর্কিটেকচারের GraphQL ভার্সন — শুরুতে এটার দরকার নেই, কিন্তু টিম/প্রজেক্ট বড় হলে জানা থাকা ভালো।

১৭. Testing GraphQL API

```
js
import { ApolloServer } from '@apollo/server';
import { typeDefs, resolvers } from '../schema.js';

test('user query সঠিক ডেটা রিটার্ন করে', async () => {
  const server = new ApolloServer({ typeDefs, resolvers });

  const response = await server.executeOperation({
    query: `query { user(id: "1") { name } }`,
  });

  expect(response.body.singleResult.data.user.name).toBe('Rafi');
});
```

Resolver-গুলো আলাদা ফাংশন বলে, এগুলোকে চাইলে সরাসরি unit test-ও করা যায় — `server.executeOperation` ছাড়াই, ঠিক যেভাবে তুমি Express controller ফাংশন আলাদা করে টেস্ট করত।

১৮. Prisma ORM + Relational Database দিয়ে GraphQL (ভবিষ্যতের প্রস্তুতি)

তুমি বলেছ পরে Prisma আর relational database ব্যবহার করবে — খুব ভালো খবর হলো, **উপরে যা শিখেছ তার প্রায় সবকিছুই অপরিবর্তিত থাকবে**। শুধু resolver-এর ভেতরের ডেটাবেজ-কলটা Mongoose থেকে Prisma Client-এ বদলায়।

Mongoose আর Prisma-র মূল পার্থক্য

বিষয়	Mongoose (MongoDB)	Prisma (PostgreSQL/MySQL)
ডেটাবেজ টাইপ	Document (NoSQL)	Relational (টেবিল, রো)

বিষয়	Mongoose (MongoDB)	Prisma (PostgreSQL/MySQL)
Schema	JS ফাইলে define	<code>.prisma</code> ফাইলে define (নিজস্ব syntax)
Relationship	Manual ref/populate	Foreign key, built-in <code>include</code>
Migration	নেই (schema-less)	Built-in migration system
Type safety	কম (runtime check)	পুরোপুরি TypeScript-generated types
Query স্টাইল	<code>Model.find()</code>	<code>prisma.model.findMany()</code>

Prisma Schema উদাহরণ

```
prisma

// schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id      Int    @id @default(autoincrement())
  name    String
  email   String @unique
  posts   Post[]
}

model Post {
  id      Int    @id @default(autoincrement())
  title   String
  content String
  author  User   @relation(fields: [authorId], references: [id])
  authorId Int
}
```

লক্ষ্য করো — MongoDB-তে `User.posts` রিলেশন তুমি manually resolver লিখে বের করতে `(Post.find({ author: id }))`। Prisma-তে relationship schema-তেই define করা থাকে `@relation`), আর migration কমান্ড দিয়ে actual ডেটাবেজ টেবিলও বানিয়ে দেয়:

```
bash
```

```
npx prisma migrate dev --name init
```

GraphQL Resolver-এ Prisma ব্যবহার

GraphQL schema (SDL) প্রায় একই থাকে — শুধু resolver ভেতরে বদলায়:

```
js

import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

export const resolvers = {
  Query: {
    users: async () => await prisma.user.findMany(),
    user: async (_, { id }) => await prisma.user.findUnique({ where: { id: Number(id) } }),
  },

  Mutation: {
    createUser: async (_, { input }) => await prisma.user.create({ data: input }),
  },

  // এখানেই Prisma-র সুবিধা: N+1 এড়াতে include ব্যবহার করা যায়
  User: {
    posts: async (parent) => {
      return await prisma.post.findMany({ where: { authorId: parent.id } });
    },
  },
};
```

একটা গুরুত্বপূর্ণ অপটিমাইজেশন: Prisma-তে DataLoader-এর বদলে (বা সাথে) `include` / `select` ব্যবহার করে এক কুয়েরিতেই relationship ডেটা আনা যায়, যদি তুমি সরাসরি resolver-এর বাইরে থেকে structure সাজাতে পারো:

```
js

user: async (_, { id }) => {
  return await prisma.user.findUnique({
    where: { id: Number(id) },
    include: { posts: true }, // একটাই কুয়েরিতে user + posts
  });
},
```

তবে এই অপটিমাইজেশন তখনই কাজ করে যদি তুমি আগে থেকেই জানো ক্লায়েন্ট posts চাইবে। GraphQL-এ ক্লায়েন্ট কী চাইবে সেটা ডায়নামিক বলে, এখনো অনেক ক্ষেত্রে DataLoader-ই বেশি নির্ভরযোগ্য সমাধান — Prisma আর DataLoader একসাথেও ব্যবহার করা যায় (`@graphql-tools/data-loader-prisma` এর মতো হেল্পার লাইব্রেরি আছে)।

কোড-ফার্স্ট অপশন: Pothos / Nexus

SDL ফাইল আলাদা করে না লিখে, পুরো schema-টাই TypeScript কোড থেকে জেনারেট করার একটা জনপ্রিয় পদ্ধতি আছে — **Pothos GraphQL** (Prisma-র সাথে ভালো ইন্টিগ্রেশন আছে)। এতে Prisma-র generated types সরাসরি GraphQL schema-তে reuse করা যায়, ডুপ্লিকেট টাইপ লেখা লাগে না। এটা একটা ভিন্ন স্টাইল — উপরের SDL-first পদ্ধতি দিয়ে আগে comfortable হও, তারপর code-first চাইলে explore করো।

সারকথা: Prisma-তে গেলে তোমার GraphQL জ্ঞানের ৯৫% একদম একই থাকবে — schema design, resolver pattern, query/mutation কনসেপ্ট, authentication, pagination সব। শুধু পরিবর্তন হবে ডেটাবেজ-লেয়ারের কোড (Mongoose query → Prisma query), আর রিলেশনাল ডেটাবেজ হওয়ায় relationship হ্যান্ডলিং আরও বেশি built-in আর type-safe হবে।

১৯. Full Mini Project: Blog API (User → Post → Comment)

পুরো জিনিসটা একসাথে দেখি — schema আর resolver, MongoDB দিয়ে।

graphql

```
# schema.graphql
type User {
  id: ID!
  name: String!
  email: String!
  posts: [Post!]!
}

type Post {
  id: ID!
  title: String!
  content: String!
  author: User!
  comments: [Comment!]!
}

type Comment {
  id: ID!
  text: String!
  author: User!
  post: Post!
}

type Query {
  posts: [Post!]!
  post(id: ID!): Post
}

input CreatePostInput {
  title: String!
  content: String!
}

type Mutation {
  createPost(input: CreatePostInput!): Post!
  addComment(postId: ID!, text: String!): Comment!
}
```

js


```
// resolvers.js
export const resolvers = {
  Query: {
    posts: async () => await Post.find(),
    post: async (_, { id }) => await Post.findById(id),
  },

  Mutation: {
    createPost: async (_, { input }, context) => {
      if (!context.user) throw new GraphQLError('লগইন আবশ্যক');
      return await Post.create({ ...input, author: context.user.id });
    },
    addComment: async (_, { postId, text }, context) => {
      if (!context.user) throw new GraphQLError('লগইন আবশ্যক');
      return await Comment.create({ text, post: postId, author: context.user.id });
    },
  },

  Post: {
    author: async (parent, _, context) => context.loaders.userLoader.load(parent.id),
    comments: async (parent) => await Comment.find({ post: parent.id }),
  },

  Comment: {
    author: async (parent, _, context) => context.loaders.userLoader.load(parent.author),
    post: async (parent) => await Post.findById(parent.post),
  },
};
```

এই একটা ছোট প্রজেক্টের ভেতর schema design, relationship resolver, mutation, auth check, আর DataLoader — সবকিছুর বাস্তব ব্যবহার দেখানো হয়েছে।

২০. শেখার Roadmap এবং পরবর্তী ধাপ

পড়েই থেমে গেলে কিছুই মাথায় থাকবে না — তাই এই ধারাবাহিকতায় হাতে-কলমে করার পরামর্শ:

1. একটা ছোট REST resource (যেমন তোমার এক্সিস্টিং কোনো প্রজেক্টের একটা feature) GraphQL-এ কনভার্ট করো — schema + resolver + Apollo Client দিয়ে
2. সেখানে relationship resolver যোগ করো, আর N+1 problem নিজে চোখে দেখো (MongoDB query log on করে), তারপর DataLoader দিয়ে ফিক্স করো
3. Authentication + authorization যোগ করো (JWT context-এ পাস করে)
4. Cursor-based pagination implement করো একটা list-এ

5. সবশেষে — Prisma দিয়ে একই প্রজেক্টের একটা নতুন ভার্সন বানিয়ে ফেলো PostgreSQL দিয়ে, যাতে দুটোর পার্থক্য নিজে অনুভব করতে পারো

অফিসিয়াল রিসোর্স:

- graphql.org/learn — কোর স্পেসিফিকেশন
- apollographql.com/docs — Apollo Server + Client ডকুমেন্টেশন
- prisma.io/docs — Prisma ORM ডকুমেন্টেশন (যখন relational DB-তে যাবে)